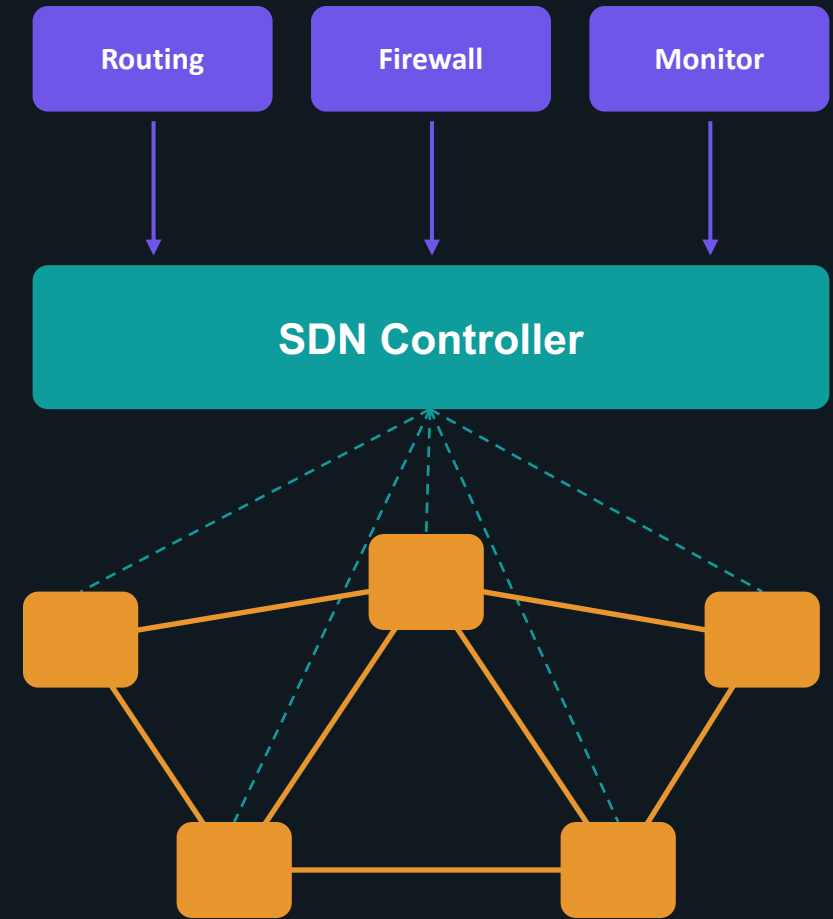


# Programming the Network

Software-Defined Networking &  
Network Functions Virtualization

**Ashok Kumar Murthy**

Principal Engineer · Aviz Networks



apps · one brain · simple, fast switches

# Hello, CEG!

What you will be able to do after these two hours

1

## Analyze

SDN and NFV architectures

2

## Identify

what the SDN data plane, control plane and OpenFlow each do

3

## Watch

a real software switch get programmed, first by hand and then by a controller

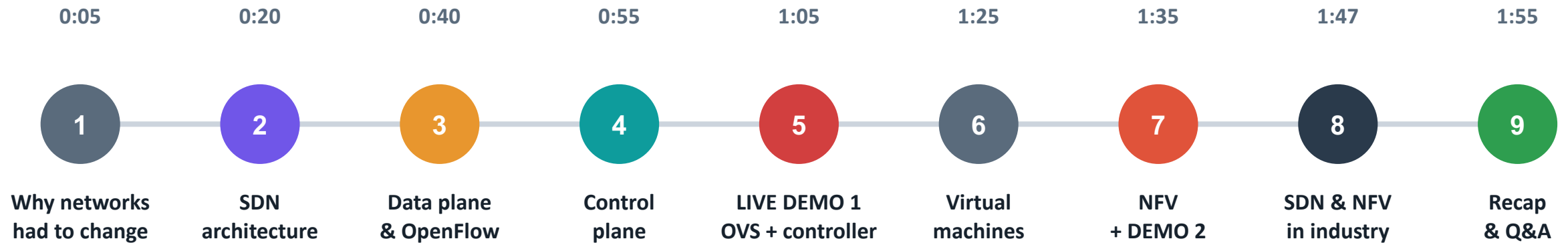
4

## Describe

how virtual machines and NFV turn network boxes into flexible software

# Today's journey

120 minutes, two live demos



# 01

## Why networks had to change

Background, motivation and evolving network requirements

APPLICATION PLANE

CONTROL PLANE

DATA PLANE

# Why do we need a switch?

BACKGROUND

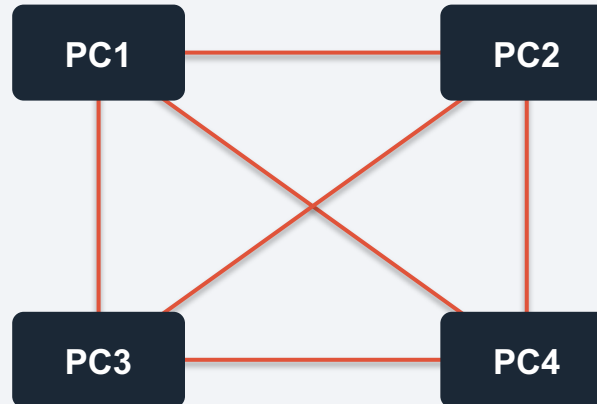
A cable is enough for two computers. It does not scale to three, let alone three thousand

Two hosts: one cable ✓



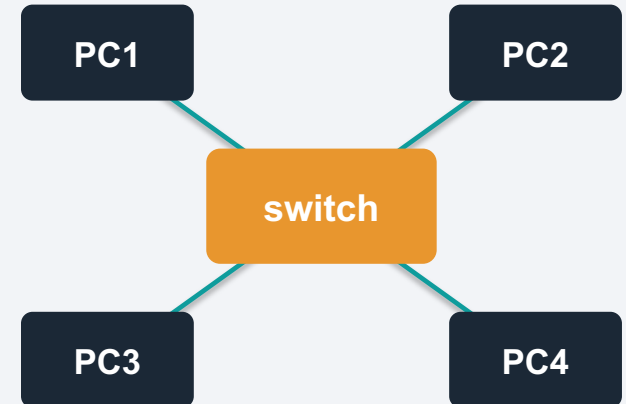
1 cable, 1 port on each PC.  
You just saw it work: ARP, then ping.

Four hosts, no switch ✗



6 cables, 3 ports on every PC.  
100 PCs: 4,950 cables, 99 ports each.

Four hosts and a switch ✓



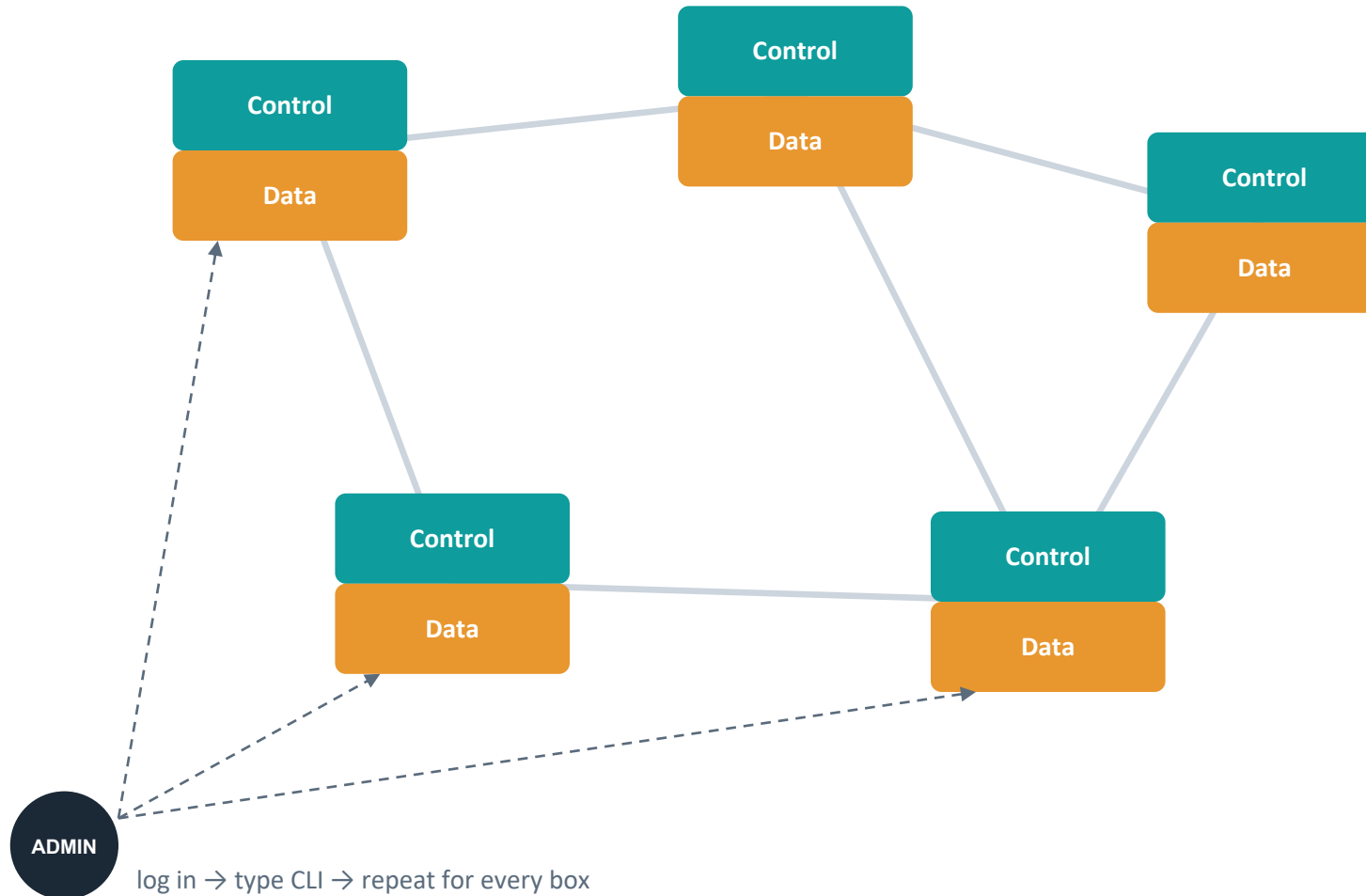
4 cables, 1 port on each PC.  
The switch learns who is where and forwards.

So the switch has to decide where every frame goes. **Who decides, and where does that logic live?**

# How networks were built

BACKGROUND

Every box is its own brain and its own muscle



## Each device decides alone

Distributed protocols (OSPF, BGP, STP) run in every box and slowly agree on paths.

## Closed and vendor-specific

Hardware, OS and features are bundled. Each vendor has its own CLI.

## Change = touch every box

A new policy means logging in to hundreds of devices, often by hand.

# Three planes inside every network device

BACKGROUND

Useful vocabulary for the rest of today

## Management plane

Configure & monitor

CLI, SNMP, NETCONF, web UI

Humans · minutes

## Control plane

Decide where packets go

Routing protocols (OSPF, BGP), ARP, STP → build tables

CPU · milliseconds

## Data (forwarding) plane

Move each packet

Look up table → forward, drop, modify, queue

ASIC · nanoseconds

who / how fast

# Why the traditional model hurts

MOTIVATION

Limitations identified by the Open Networking Foundation (ONF)



## Static & complex

Adding or moving a device touches many switches, ACLs, VLANs and QoS settings. Networks became brittle and change was feared.



## Inconsistent policies

A security policy must be pushed box by box. One missed device creates a hole or an outage.



## Inability to scale

Cloud and data-center growth outpaced manual, per-device operations and oversubscription planning.



## Vendor dependence

New features wait for the vendor's release cycle, often years. Operators cannot innovate on their own network.

# Evolving network requirements

## REQUIREMENTS

Demand exploded, and traffic changed shape

### Cloud computing

Millions of tenants creating networks on demand, in seconds, through an API.

### Big data & AI

Massive east-west flows between servers; AI training clusters need lossless, high-bandwidth fabrics.

### Mobile traffic

Smartphones and 4G/5G: bursty, always-on traffic from billions of devices.

### Internet of Things

Sensors, cameras and vehicles: huge numbers of small endpoints needing policy.

### Server virtualization

VMs and containers move between hosts; the network must follow them automatically.

### Video & streaming

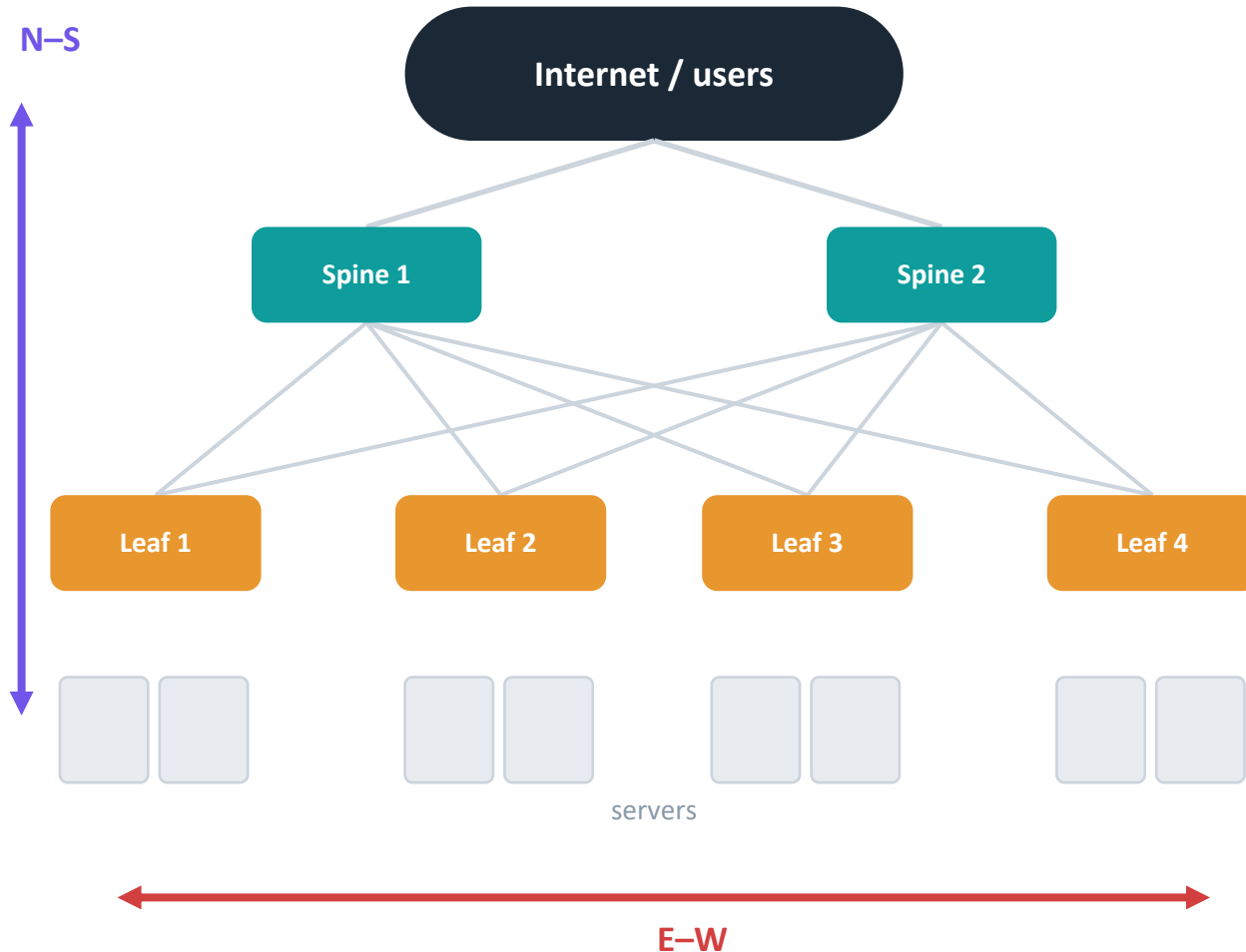
The majority of Internet bytes; needs QoS and capacity close to users.

**Supply side kept up:** Ethernet went from 1 Gb/s (1998) to 800 Gb/s (IEEE 802.3df, 2024). Hardware was fast enough; the bottleneck became how networks are controlled.

# Traffic changed direction

## REQUIREMENTS

From north–south (client ↔ server) to east–west (server ↔ server)



## Then

Most traffic went between a user and one server. A tree of access, aggregation and core switches was fine.

## Now

One web request fans out to hundreds of micro-services, databases and caches. Most bytes stay inside the data center.

## So

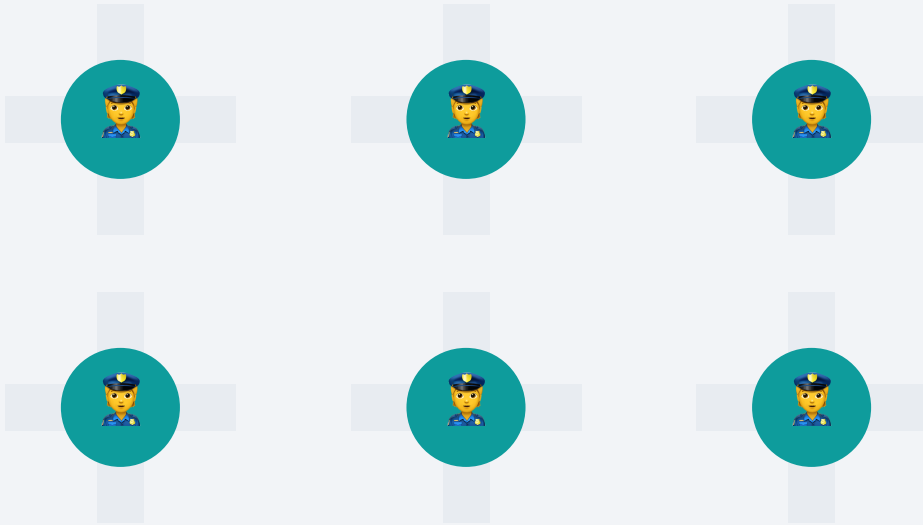
Leaf-spine fabrics, many equal-cost paths, and software that can program all of them together.

# The big idea: separate the brain from the muscle

MOTIVATION

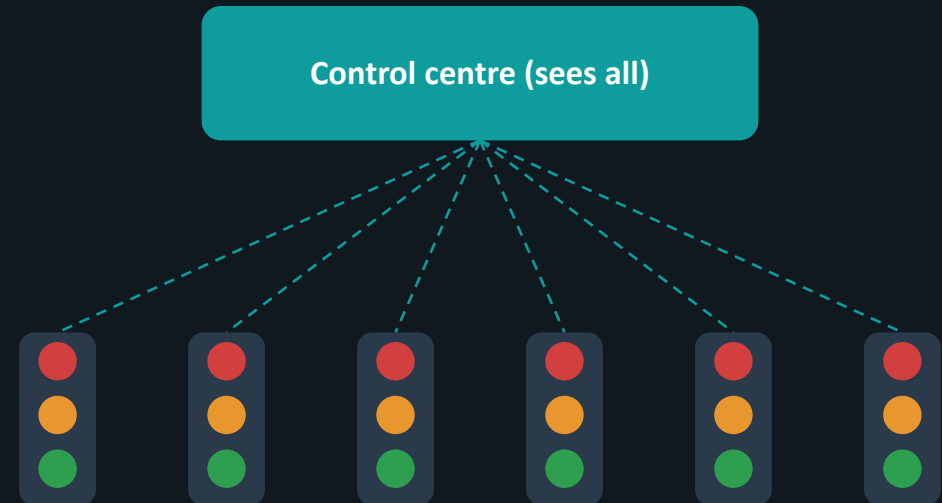
An analogy

## Traditional: a policeman at every junction



Each decides alone, sees only his junction, and does not know about the jam three signals away.

## SDN: an integrated traffic control centre

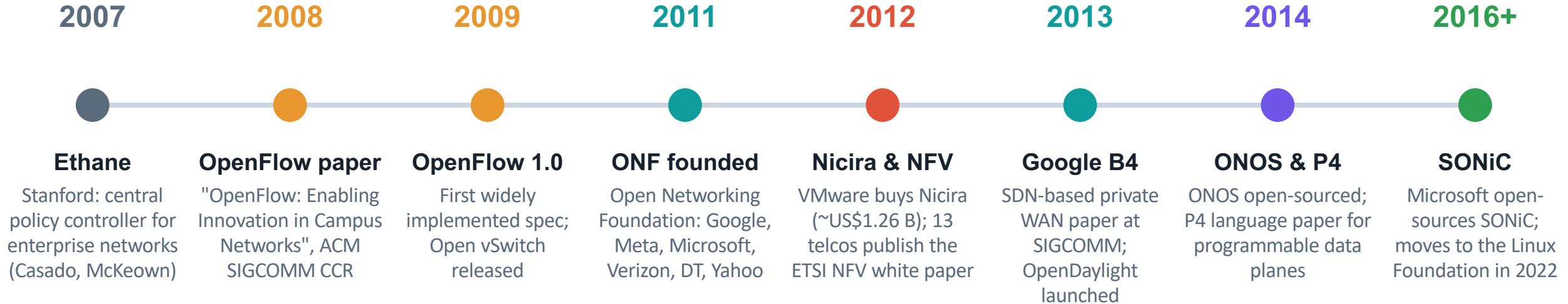


Signals just follow rules. One centre with the whole city's view programs them all, and can change the plan in seconds.

# A short history of SDN & NFV

BACKGROUND

From a Stanford PhD project to how the cloud runs



**Two separate stories:** SDN came from universities and cloud companies asking "who controls the network?". NFV came from telecom operators asking "why is every function a separate box?".

# 02

## SDN architecture

Planes, interfaces and the life of a packet

APPLICATION PLANE

CONTROL PLANE

DATA PLANE

# What is Software-Defined Networking?

“The physical separation of the network control plane from the forwarding plane, where a control plane controls several devices.”

Open Networking Foundation

1

## Separate control from data

Switches only forward.  
Decisions are made elsewhere.

2

## Logically centralized control

One controller (or cluster) holds a global view of the network.

3

## Open interfaces

Standard APIs between layers: southbound (e.g. OpenFlow) and northbound (e.g. REST).

4

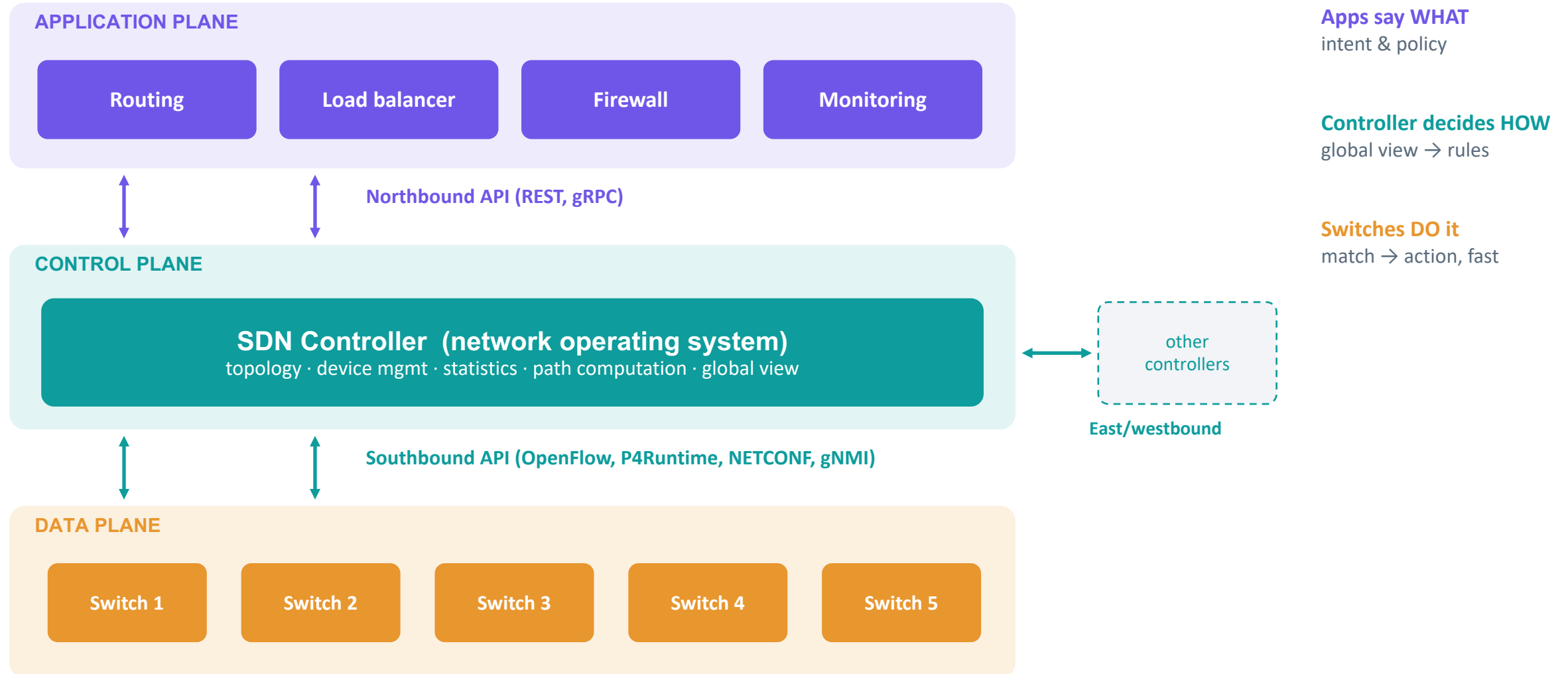
## Programmability

Network behaviour is defined by applications, i.e. software, rather than per-box configuration.

# The SDN architecture

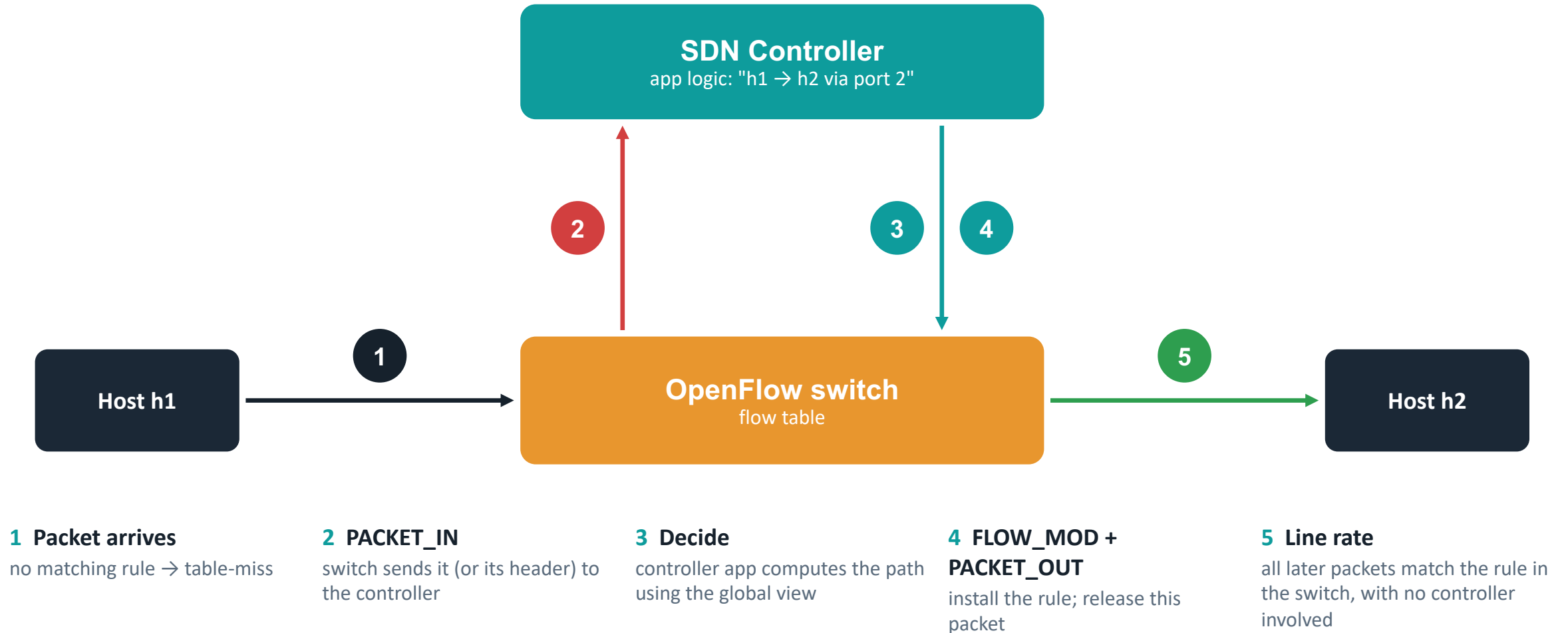
## SDN ARCHITECTURE

Three planes, connected by open APIs



# Life of a packet in an SDN

Reactive flow set-up: the first packet asks, the rest just flow



# Reactive vs proactive flow installation

## Reactive

Rules installed when the first packet of a flow arrives.

- Flexible: per-flow decisions
- Small flow tables
- First-packet delay (controller round-trip)
- Controller load grows with new flows
- Controller down → new flows fail

## Proactive

Rules pushed in advance, before traffic arrives.

- No set-up delay
- Survives controller outage
- Predictable, scales to data centers
- Needs larger tables / wildcards
- Must know the policy up front

# Traditional networking vs SDN

|                            | Traditional                     | Software-Defined                             |
|----------------------------|---------------------------------|----------------------------------------------|
| <b>Control plane</b>       | Distributed, inside every box   | Logically centralized controller             |
| <b>Network view</b>        | Each device sees its neighbours | Controller sees the whole network            |
| <b>Configuration</b>       | Per-device CLI, manual          | Programmatic, via APIs & automation          |
| <b>Hardware / software</b> | Bundled by one vendor           | Decoupled; open, commodity hardware possible |
| <b>New features</b>        | Wait for vendor release         | Write an application                         |
| <b>Policy consistency</b>  | Hard, drift between boxes       | Enforced centrally                           |
| <b>Failure domain</b>      | Robust, no single brain         | Controller must be made highly available     |
| <b>Typical examples</b>    | Campus & legacy enterprise      | Cloud DCs, Google B4, SD-WAN, 5G             |

# 03

## Data plane & OpenFlow

Flow tables, match–action, and the protocol between switch and controller

APPLICATION PLANE

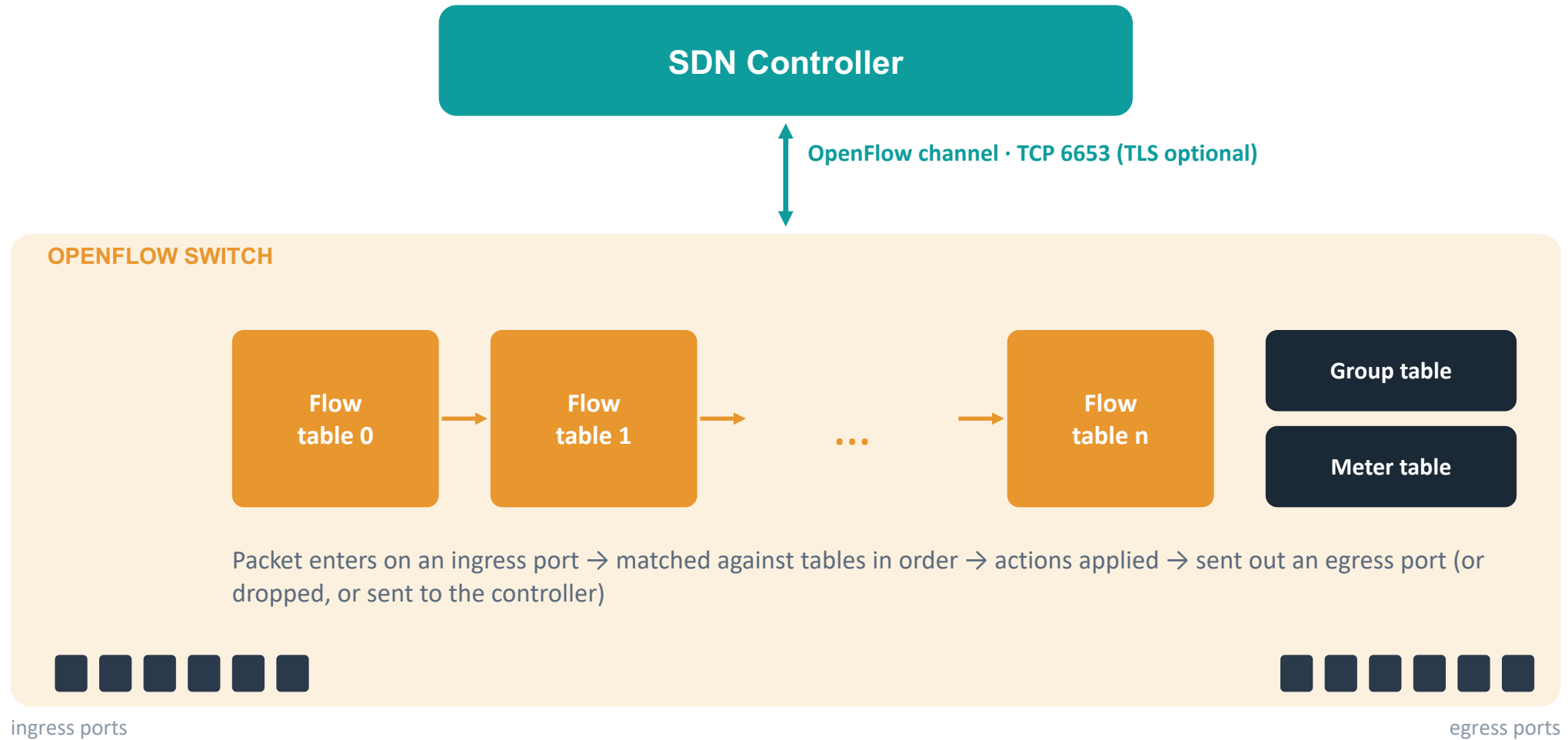
CONTROL PLANE

DATA PLANE

# Inside an OpenFlow switch

DATA PLANE

The data plane is a pipeline of tables



# Anatomy of a flow entry

OPENFLOW

Every rule is a match plus what to do

| Match fields                                                                                               | Priority                     | Counters                             | Instructions                              | Timeouts              | Cookie                           | Flags                   |
|------------------------------------------------------------------------------------------------------------|------------------------------|--------------------------------------|-------------------------------------------|-----------------------|----------------------------------|-------------------------|
| Header values and ingress port to compare, with wildcards allowed. Up to 40+ fields (OXM) in OpenFlow 1.3. | Highest-priority match wins. | Packets and bytes matched; duration. | Actions to apply, or go to another table. | Idle and hard expiry. | Opaque ID set by the controller. | e.g. notify on removal. |

## Common match fields → actions

**Match:** in\_port · eth\_src/dst · eth\_type · vlan\_id · ip\_src/dst · ip\_proto · tcp/udp ports · ...

**Action:** output:port · drop · flood · CONTROLLER · set-field · push/pop VLAN · group · meter

# A flow table you can read

OPENFLOW

What the switch in our demo will hold

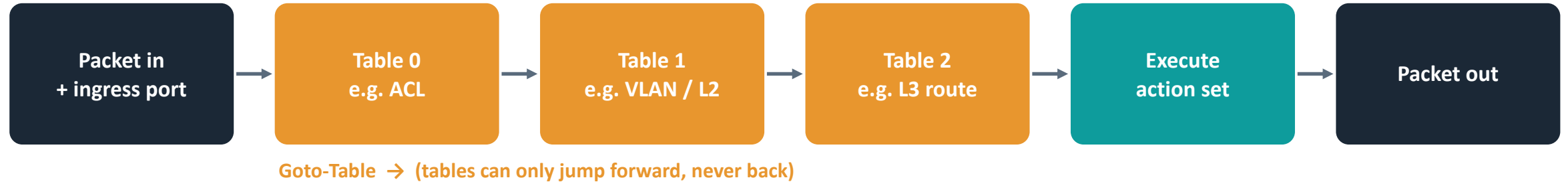
| Priority | Match                                  | Actions      | Meaning                                       |
|----------|----------------------------------------|--------------|-----------------------------------------------|
| 300      | icmp, nw_src=10.0.0.1, nw_dst=10.0.0.2 | drop         | Firewall: h1 may not ping h2                  |
| 200      | ip, nw_dst=10.0.0.2                    | output:s1-h2 | Deliver to h2                                 |
| 200      | ip, nw_dst=10.0.0.1                    | output:s1-h1 | Deliver to h1                                 |
| 100      | arp                                    | FLOOD        | Let ARP find neighbours                       |
| 0        | (anything else: table-miss)            | CONTROLLER   | Ask the brain (when a controller is attached) |

**Try it:** h1 pings h2. The packet matches rules 300 and 200. Which wins? And will h1 still be able to open h2's web page?

# Multiple tables: the OpenFlow pipeline

OPENFLOW

Since OpenFlow 1.1: match in stages instead of one giant table



## Apply-Actions

Do these actions now (e.g. push a VLAN tag) before moving on.

## Clear-Actions

Empty the action set.

## Write-Metadata

Carry a tag between tables (e.g. which tenant or VRF).

## Write-Actions

Add actions to the packet's action set, executed at the end of the pipeline.

## Goto-Table

Continue matching in a later table.

## Meter

Send through a rate limiter first (OpenFlow 1.3).

# Group and meter tables

OPENFLOW

Reusable actions and built-in QoS

## Group types

**ALL**

**Execute every bucket**

Multicast, broadcast, flooding

**SELECT**

**Execute one bucket (hash)**

Load balancing, ECMP

**INDIRECT**

**Single shared bucket**

Many routes share one next hop; update once

**FAST-FAILOVER**

**First live bucket**

Instant backup path when a port goes down

## Meters (OpenFlow 1.3+)

Measure a flow's rate and act when it exceeds a band:

**$\leq 10$  Mb/s**

forward normally

**$> 10$  Mb/s**

remark DSCP (lower priority)

**$> 50$  Mb/s**

drop

e.g. rate-limit a guest Wi-Fi VLAN

# OpenFlow protocol messages

OPENFLOW

How the controller and the switch talk

## Controller → switch

### FEATURES

who are you? (datapath ID, ports)

### FLOW\_MOD

add / modify / delete flows

### PACKET\_OUT

send this packet out a port

### MULTIPART

read stats: flows, ports, tables

### BARRIER

finish previous messages first

### ROLE

master / slave / equal controller

## Asynchronous (switch → controller)

### PACKET\_IN

table-miss or explicit send-to-controller

### FLOW\_REMOVED

a flow timed out or was deleted

### PORT\_STATUS

link went up or down

## Symmetric (either side)

### HELLO

version negotiation at start

### ECHO

keep-alive / latency check

### ERROR

something failed

### EXPERIMENTER

vendor extensions

# OpenFlow versions, and what came next

OPENFLOW

## OF 1.0

2009

Single table, 12  
match fields

## OF 1.1

2011

Multiple tables,  
groups, MPLS

## OF 1.2

2011

Extensible match  
(OXM), IPv6

## OF 1.3

2012

Meters, per-flow  
counters: the  
most deployed

## OF 1.4

2013

Bundles, optical  
ports

## OF 1.5

2014

Egress tables,  
packet-type  
aware

## Beyond OpenFlow: P4

OpenFlow fixes which headers a switch understands.  
P4 lets you program the parser and pipeline itself, and  
invent new headers.

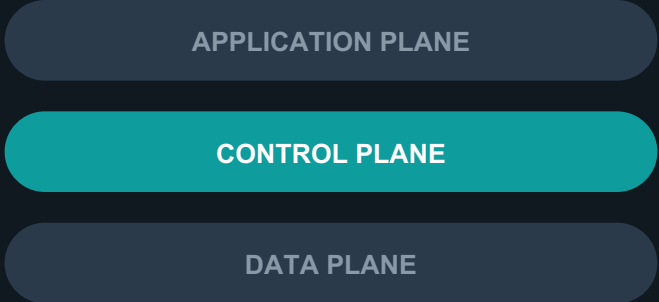
## Today's southbound mix

P4Runtime for programmable chips · gNMI / OpenConfig for  
config & streaming telemetry · NETCONF/YANG · OVSDB for  
Open vSwitch

# 04

## Control plane architecture

Inside the SDN controller, the network operating system



≈ 10 MIN

# What an SDN controller does

CONTROL PLANE

A network operating system: core services exposed to apps

Northbound API → apps: routing, load balancing, security, monitoring

## Topology manager

Discovers switches, links, hosts (LLDP)

## Device manager

Tracks switches, ports and capabilities

## Statistics manager

Collects counters: flows, ports, tables

## Shortest-path forwarding

Computes routes over the topology graph

## Notification manager

Events: link down, new host, flow removed

## Security mechanisms

Isolation between apps; TLS; authorization

Southbound API → OpenFlow · OVSD · NETCONF · P4Runtime · gNMI

## How topology discovery works

- Controller sends an LLDP packet out every switch port (PACKET\_OUT).
- The neighbour switch receives it and, having no rule, sends it back up (PACKET\_IN).
- "Sent from S1 port 2, arrived at S3 port 1" → that is a link!
- Repeat for every port and you have the full graph.

# Northbound: telling the network WHAT, not HOW

CONTROL PLANE

Apps talk to the controller through APIs, often REST

## Intent

"Connect host A to host B, avoid link L3"

## Path / service

"Use path S1 → S4 → S2 with 100 Mb/s"

## Flow rules

"On S1: match ip\_dst=10.0.0.2 → output:3"

**Higher abstraction = less the app needs to know.**

The controller compiles intents into flow rules, and re-computes them automatically if a link fails.

```
northbound REST

# Ask ONOS for all flows (REST, northbound)
$ curl -u onos:rocks \
  http://localhost:8181/onos/v1/flows

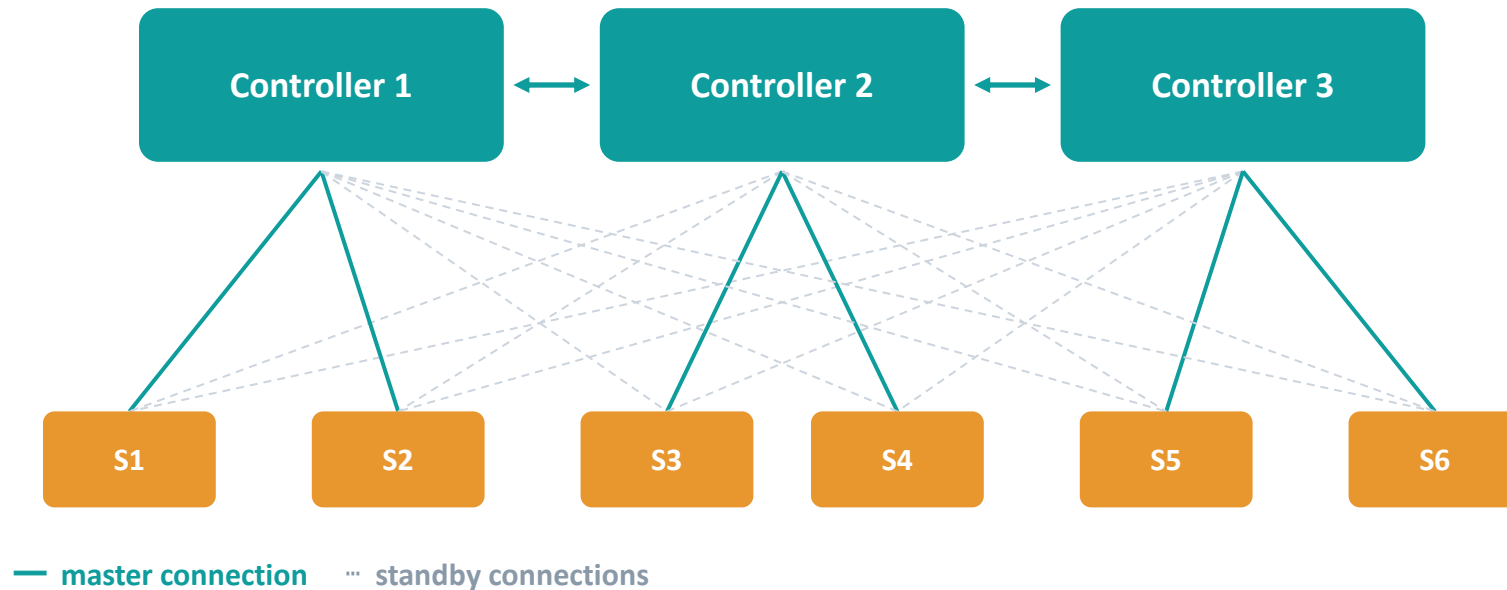
# Submit an intent: connect two hosts
$ curl -u onos:rocks -X POST \
  -H "Content-Type: application/json" \
  http://localhost:8181/onos/v1/intents \
  -d '{"type": "HostToHostIntent",
      "appId": "org.onosproject.cli",
      "one": "<h1-id>", "two": "<h2-id>"}'
```

# Scaling the brain: distributed control

CONTROL PLANE

"Logically centralized" does not mean one server

cluster shares state (consistent store, e.g. Raft consensus)



## High availability

If controller 2 dies, S3/S4 fail over to a standby. OpenFlow ROLE messages handle master/slave.

## Scale-out

Add controllers as the network grows; each masters a share of switches.

## Consistency

All instances must agree on topology & flows: a distributed-systems problem.

## Hierarchy / federation

Large WANs use per-region controllers plus a global one (east-west APIs).

**ONOS example:** runs as a cluster (typically 3 or 5 nodes) with a Raft-based distributed store (Atomix). Built for carrier-grade availability.

# The controller landscape

CONTROL PLANE

Open-source controllers you will meet

| Controller          | Language     | From                          | Known for                                          |
|---------------------|--------------|-------------------------------|----------------------------------------------------|
| <b>NOX / POX</b>    | C++ / Python | Nicira, Stanford (2008)       | The first controllers; POX still used for teaching |
| <b>Ryu</b>          | Python       | NTT                           | Simple, readable apps; basis of Faucet             |
| <b>Floodlight</b>   | Java         | Big Switch Networks           | Early enterprise-grade OpenFlow controller         |
| <b>OpenDaylight</b> | Java         | Linux Foundation (2013)       | Modular, multi-protocol (NETCONF, BGP-LS, PCEP)    |
| <b>ONOS</b>         | Java         | ON.Lab/ONF → Linux Foundation | Carrier-grade, clustered, intents, GUI             |
| <b>Faucet</b>       | Python       | Open source community         | Production OpenFlow; config in YAML                |

**Commercial SDN:** VMware NSX (network virtualization), Cisco ACI (policy-based DC fabric), cloud providers' own controllers (AWS, Azure, Google VPC networking).

# SDN at planet scale: Google B4

CONTROL PLANE

The private WAN that links Google's data centers

**~100%**

utilization on many links, instead of the usual 30–40%

**~70%**

average link utilization over long periods

**2–3×**

efficiency improvement vs. a standard WAN

## How they did it

- Built their own switches from merchant silicon, controlled via OpenFlow.
- A central traffic-engineering (TE) server computes paths for all applications, knowing their priorities and demand.
- Bulk traffic (copies, backups) fills the gaps; latency-sensitive traffic gets priority. Paths shift in seconds.

LIVE DEMO 1

# Build an SDN on a laptop

Open vSwitch + containerlab + ONOS, all running in containers on this MacBook

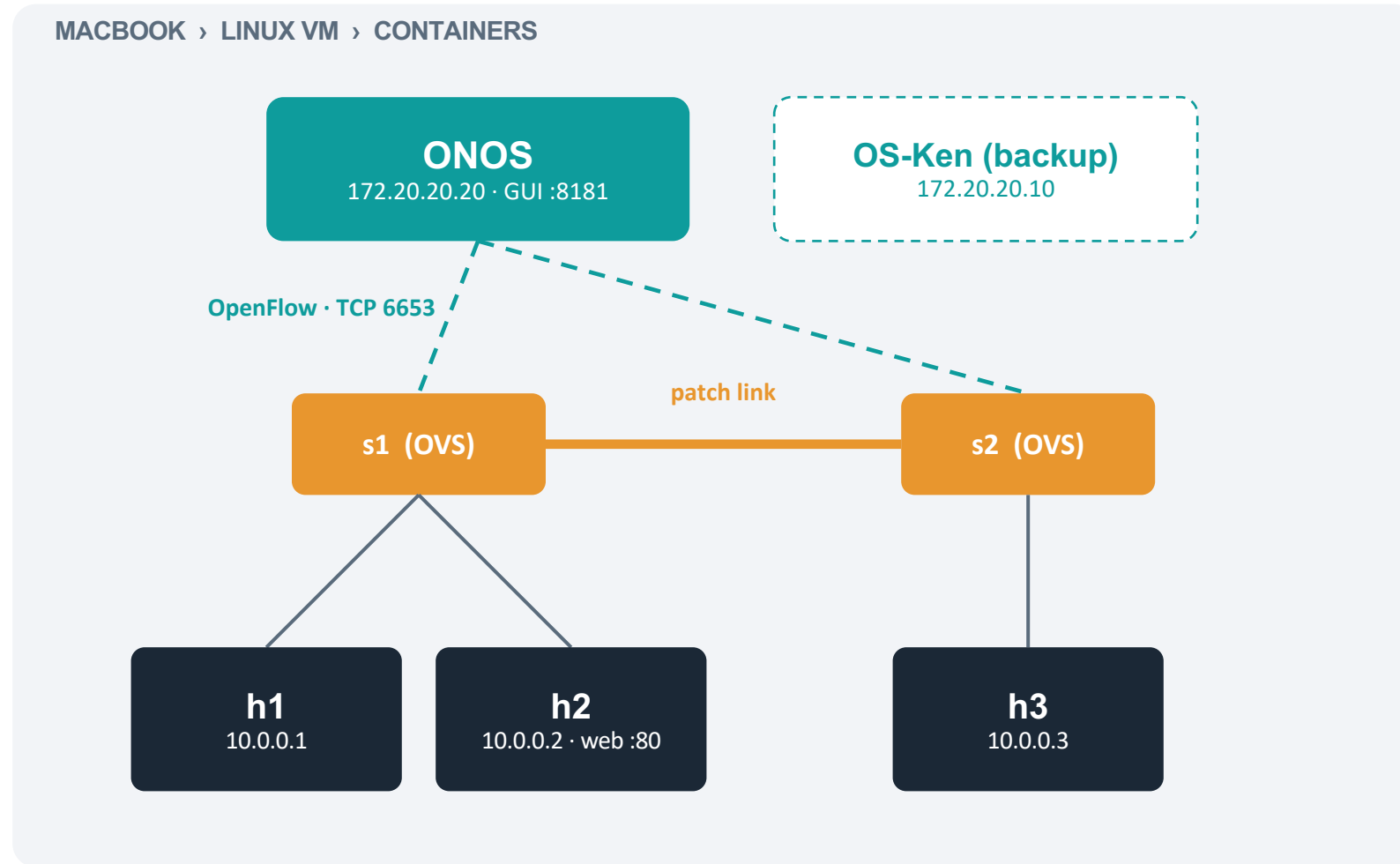
```
ashok@sdn-lab: ~/sdn-demo

$ sudo containerlab deploy -t sdn.clab.yml
$ sudo ovs-ofctl dump-flows s1
# (empty: a switch with no brain)
$ ... add-flow s1 "ip,nw_dst=10.0.0.2,
    actions=output:s1-h2"
$ sudo ovs-vsctl set-controller s1 \
    tcp:172.20.20.20:6653
# ONOS takes over
```

# The lab

DEMO 1

Three hosts, two software switches, one controller, all defined in a YAML file



## containerlab

Topology as code: nodes + links in YAML; one command deploys it

## Open vSwitch

The OpenFlow switch used inside clouds; here as s1 and s2

## ONOS 2.7

Open-source SDN controller: reactive forwarding app, GUI, REST API

## Containers

Each host is a tiny Linux with its own network stack

# What we will do: four steps

DEMO 1

1

## A switch with no brain

Empty flow table,  
fail\_mode=secure.

**Expect: ping h1 → h2 fails**

2

## You are the controller

Write 3 flow rules by hand  
with ovs-ofctl.

**Expect: ping works, counters rise**

3

## A firewall in one line

Add a priority-300 rule that  
drops ICMP h1 → h2.

**Expect: ping blocked, web still works**

4

## Hand over to ONOS

Point s1 and s2 at the  
controller.

**Expect: flows appear by themselves; topology in the GUI**

**Safety net:** scripts/reset.sh returns to step 1 in 2 seconds · CTRL=osken switches to the backup controller · a recorded video is on the USB stick

# Demo 1 commands: steps 1–3

DEMO 1

Take a photo: you can run all of this on your own laptop

demo1.sh · steps 1-3

```
# 1 no brain: empty table, nothing forwards
$ sudo ovs-ofctl -O OpenFlow13 dump-flows s1
$ docker exec clab-sdn-h1 ping -c2 10.0.0.2 # fails
# 2 be the controller
$ sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=100,arp,actions=FLOOD"
$ sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=200,ip,nw_dst=10.0.0.1,actions=output:s1-h1"
$ sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=200,ip,nw_dst=10.0.0.2,actions=output:s1-h2"
$ docker exec clab-sdn-h1 ping -c3 10.0.0.2 # works
# 3 firewall = higher-priority rule
$ sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=300,icmp,nw_src=10.0.0.1,nw_dst=10.0.0.2,actions=drop"
$ docker exec clab-sdn-h1 curl -s http://10.0.0.2 # web still allowed
$ sudo ovs-appctl ofproto/trace s1 in_port=s1-h1,icmp,nw_src=10.0.0.1,nw_dst=10.0.0.2
```

# Demo 1 commands: step 4

DEMO 1

The controller takes over

```
demo1.sh · step 4

# clean slate, then point both switches at ONOS
$ sudo ovs-ofctl -O OpenFlow13 del-flows s1
$ sudo ovs-vsctl set-controller s1 tcp:172.20.20.20:6653
$ sudo ovs-vsctl set-controller s2 tcp:172.20.20.20:6653
$ sudo ovs-vsctl show | grep is_connected

# traffic now triggers PACKET_IN -> FLOW_MOD
$ docker exec clab-sdn-h1 ping -c4 10.0.0.3
$ sudo ovs-ofctl -O OpenFlow13 dump-flows s1

# northbound REST API
$ curl -u onos:rocks \
  localhost:8181/onos/v1/devices
```

## Watch for

- `is_connected`: true on both switches
- First ping may lose a packet while the path is set up (reactive)
- New flows in s1 with ONOS's cookie and priorities
- GUI: localhost:8181/onos/ui. Press H to show hosts
- Nobody told ONOS about the s1–s2 link: it discovered it with LLDP

# What you just saw, mapped to the architecture

DEMO 1

## Application plane

ONOS reactive-forwarding app · GUI · REST calls with curl

Northbound API (REST on :8181)

## Control plane

First YOU typing ovs-ofctl, then the ONOS controller

Southbound API (OpenFlow 1.3 on :6653)

## Data plane

Open vSwitch s1 and s2: match → count → act, no decisions of its own

Flow tables (dump-flows)

# 05

## Virtual machines

One physical server, many isolated machines, and why networks care

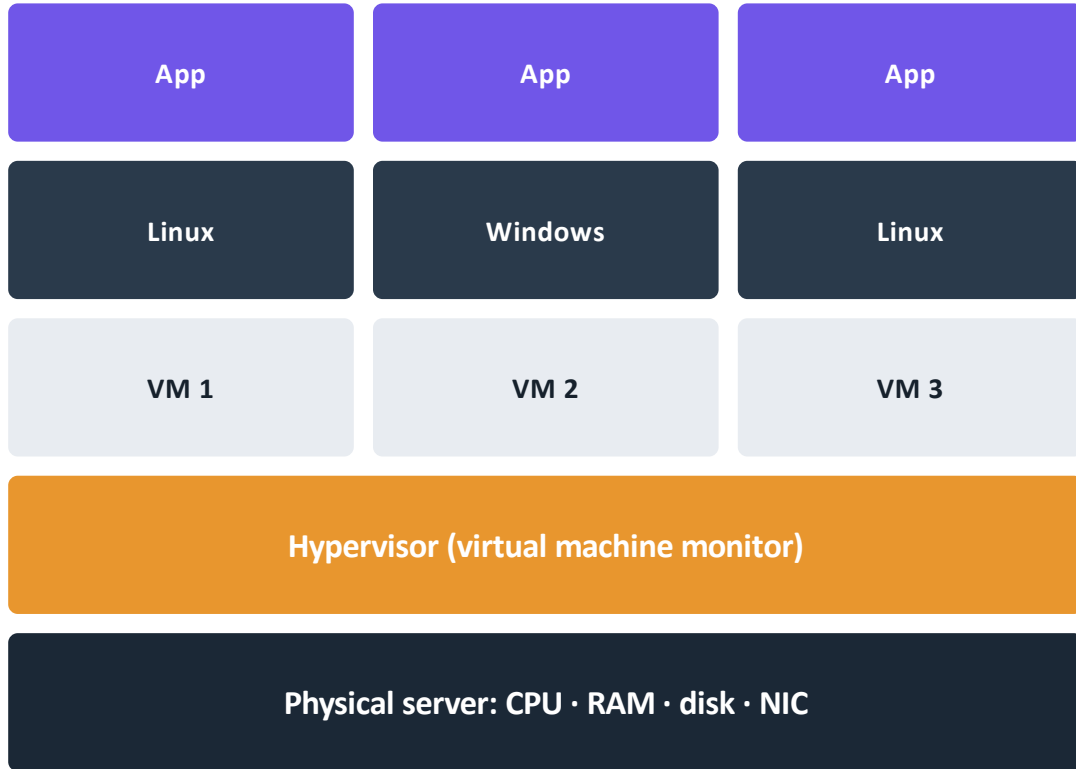
APPLICATION PLANE

CONTROL PLANE

DATA PLANE

# Virtualization: many machines in one box

VIRTUAL MACHINES



Each VM believes it owns real hardware. The hypervisor shares out the real CPU, RAM and devices.

1

## Consolidation

Run 10–30 workloads on one server instead of 10–30 servers.

2

## Isolation

A crash or compromise in one VM does not affect the others.

3

## Agility

Create, clone, snapshot or delete a machine in minutes, by API.

4

## Mobility

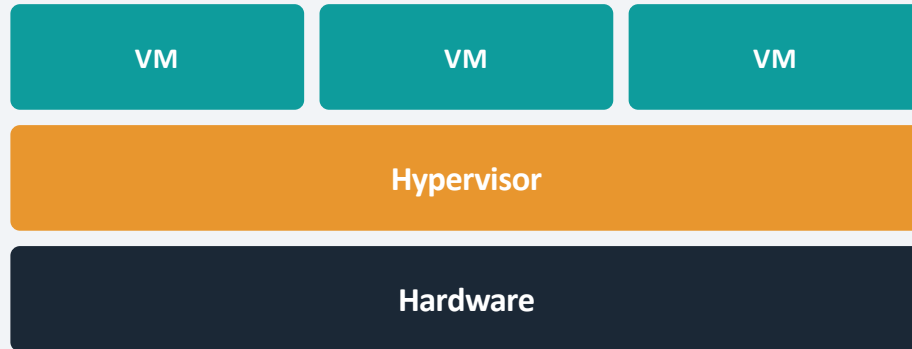
Live-migrate a running VM to another host with no downtime.

# Two kinds of hypervisor

VIRTUAL MACHINES

## Type 1 · bare-metal

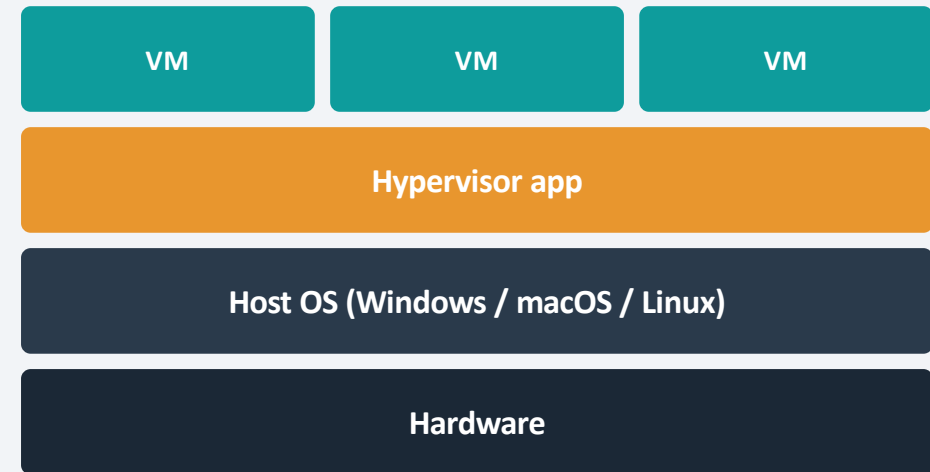
Hypervisor runs directly on the hardware



VMware ESXi · KVM (Linux) · Xen · Microsoft Hyper-V  
Used in: data centers, clouds, NFV infrastructure

## Type 2 · hosted

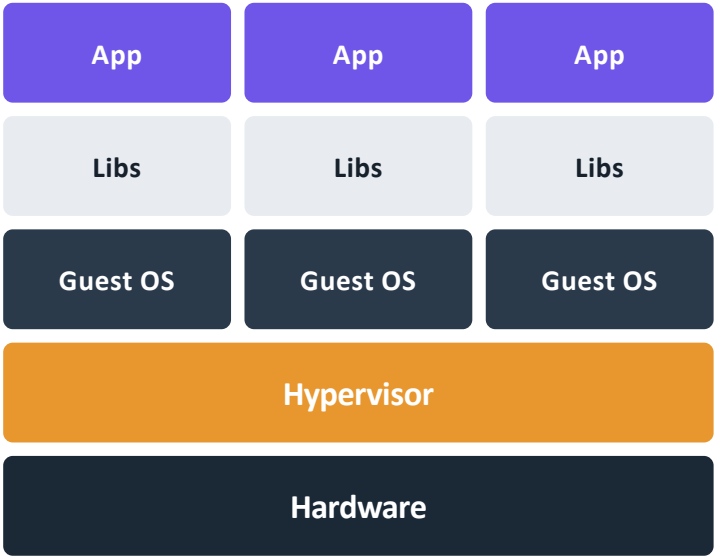
Hypervisor runs as an app on a normal OS



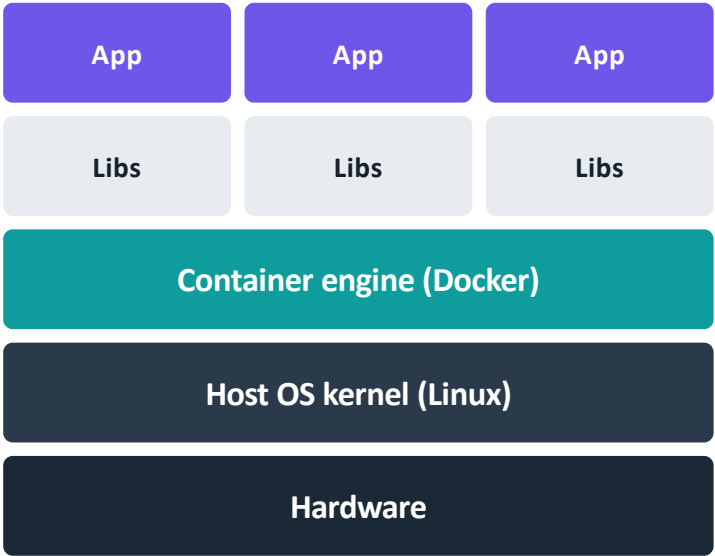
VirtualBox · VMware Workstation / Fusion · Parallels · UTM  
Used in: laptops, labs, development

# Virtual machines vs containers

Containers share the host kernel: lighter, faster, less isolated



Virtual machines



Containers

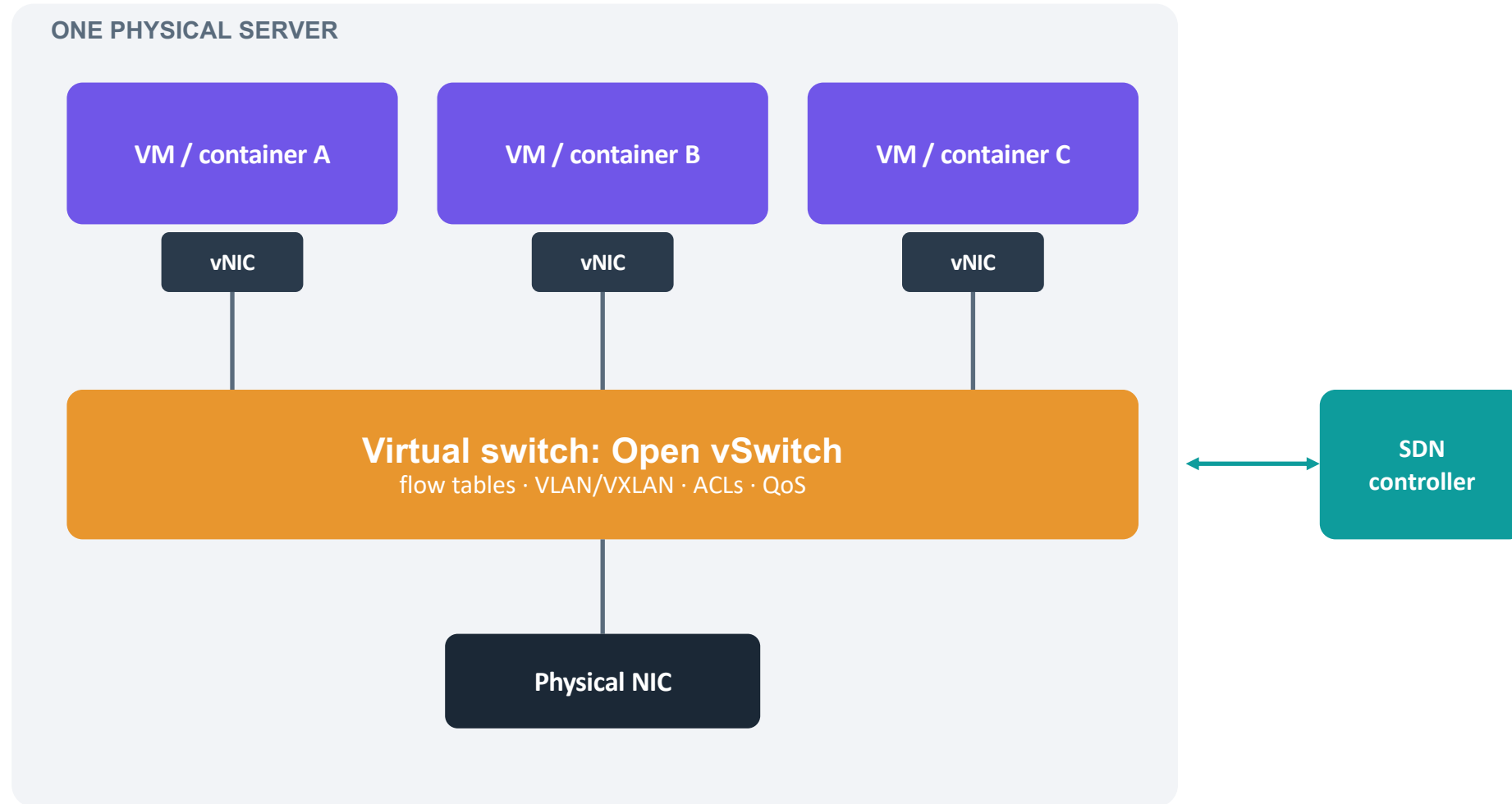
|           | VM                  | Container          |
|-----------|---------------------|--------------------|
| Boots in  | 10s of seconds      | < 1 second         |
| Size      | GBs                 | MBs                |
| Isolation | Strong (own kernel) | Process-level      |
| Other OS? | Yes                 | No (shared kernel) |

**Why this matters today:** our demo lab is a set of containers. Each "host" and "router" is a container with its own network stack, wired together with virtual links. You can build a 10-node network on a laptop in seconds.

# The network inside a server

VIRTUAL MACHINES

The first switch a packet meets is often software



## Open vSwitch

Created at Nicira (2009), now a Linux Foundation project.

### Speaks OpenFlow + OVSDb

Default virtual switch in OpenStack; widely used in KVM, Kubernetes CNIs and NFV platforms.

**The star of Demo 1.**

# 06

## Network Functions Virtualization

From dedicated appliances to software on standard servers

APPLICATION PLANE

CONTROL PLANE

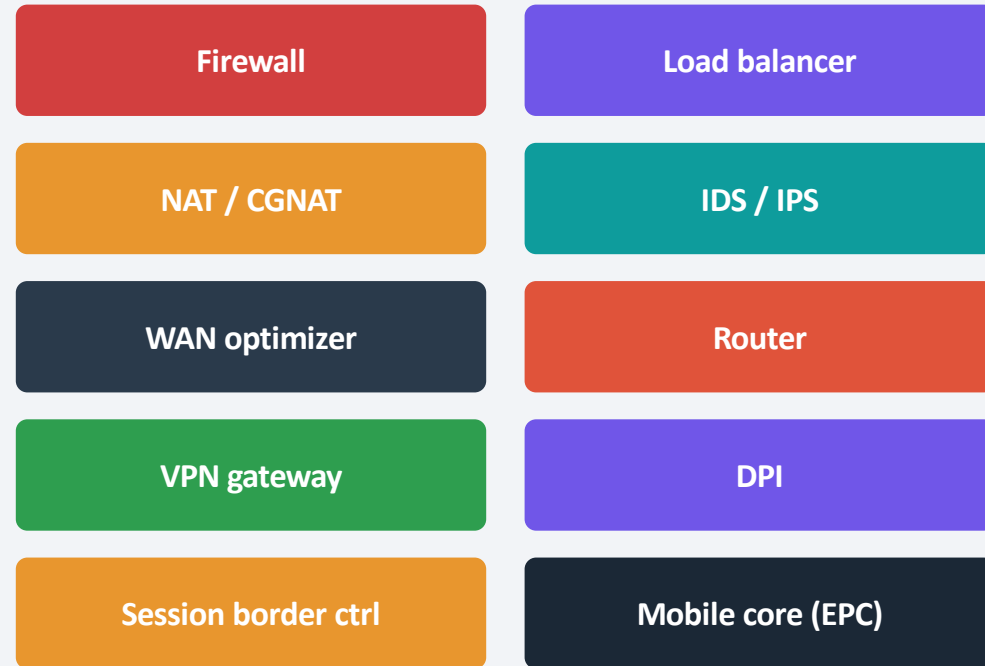
DATA PLANE

# The problem: appliance sprawl

NFV

Every network function used to be its own box

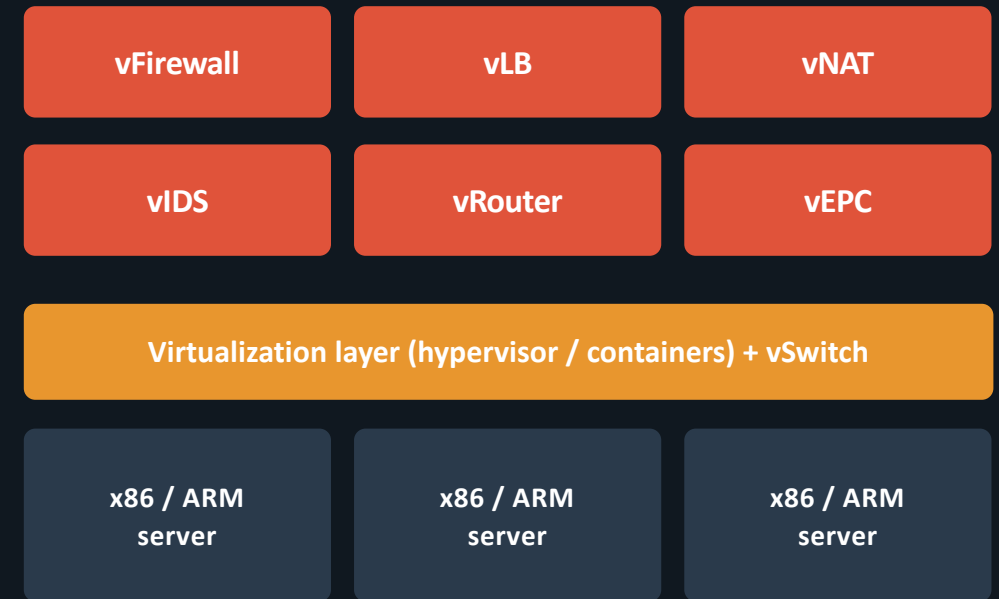
## Before: purpose-built hardware



Different vendor, OS, spares and training for each. Months to deploy.



## After: software on standard servers



Same hardware for every function. New service = deploy software, in minutes.

# NFV: the idea and its key concepts

**NFV** decouples network functions from proprietary hardware, so they run as software on industry-standard servers, storage and switches.

Launched by 13 telecom operators in the ETSI NFV white paper, October 2012

## VNF

### Virtual Network Function

A network function (firewall, router, EPC...) implemented in software, running in VMs or containers.

## NFVI

### NFV Infrastructure

The compute, storage and network hardware plus the virtualization layer that VNFs run on.

## MANO

### Management & Orchestration

Deploys, scales, heals and retires VNFs and the resources they use.

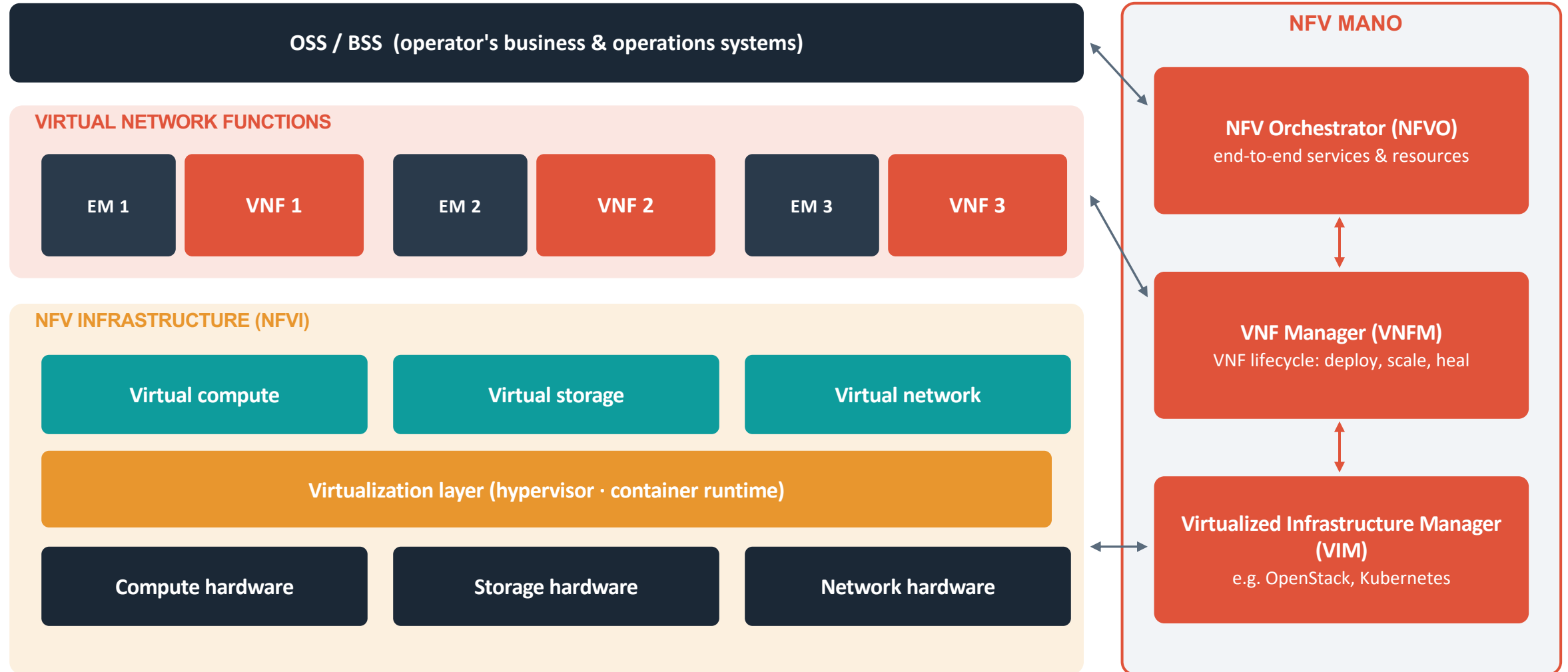
## VNF-FG

### Forwarding graph / service chain

The ordered path traffic takes through several VNFs, e.g. firewall → IDS → NAT.

# The ETSI NFV architectural framework

NFV

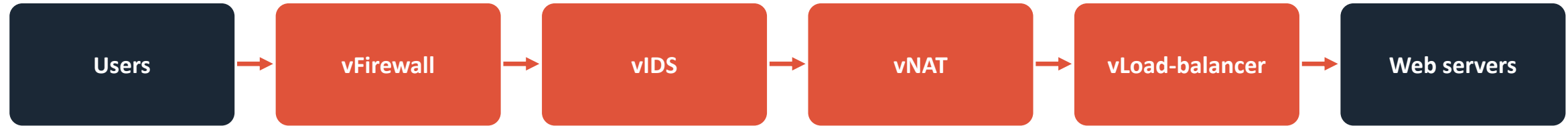


Based on ETSI GS NFV 002, "NFV Architectural Framework" · EM = element management

# Service function chaining

NFV

Traffic visits a sequence of VNFs, and software defines the order



Enterprise chain (north-south traffic)



A different, shorter chain for a different class of traffic, on the same servers

**SDN + NFV together:** the SDN controller installs flow rules that steer each traffic class through its chain. Add a VNF, re-order or scale out: change the rules, not the cables.

# NFV benefits

NFV

Why operators adopted it



## Lower CAPEX

Commodity servers instead of specialized appliances; buy capacity as you grow.



## Lower OPEX

Less power, space and spares; one team and one platform to run.



## Faster time to market

New service = new software, deployed in minutes, not months.



## Elastic scaling

Add or remove VNF instances as demand changes, e.g. during festival sales.



## Open ecosystem

Software vendors, startups and open source can compete; less lock-in.



## Targeted services

Deploy functions only where and when needed: by region, customer or time.



## Multi-tenancy

Share one infrastructure among many customers or services, isolated.



## Resilience & automation

Automatic healing and migration of failed functions.

# NFV requirements and challenges

What an NFV platform must get right

## 01 Portability & interoperability

VNFs must run on any vendor's standard hardware and NFVI.

## 02 Performance

Software must approach hardware speed: DPDK, SR-IOV, CPU pinning, SmartNICs/DPUs.

## 03 Coexistence & migration

Must work alongside legacy boxes during a years-long transition.

## 04 Management & orchestration

Consistent MANO across VNFs from many vendors.

## 05 Automation

Everything by API; manual steps do not scale.

## 06 Security & resilience

Shared infrastructure is a bigger attack surface; failures must be contained.

## 07 Network stability

Moving and scaling VNFs must not disrupt live traffic.

## 08 Simplicity

Operations must get simpler, not more complex, than the boxes they replace.

## 09 Integration

Hardware, hypervisors and VNFs from different vendors must integrate cleanly.

# SDN and NFV: different ideas, better together

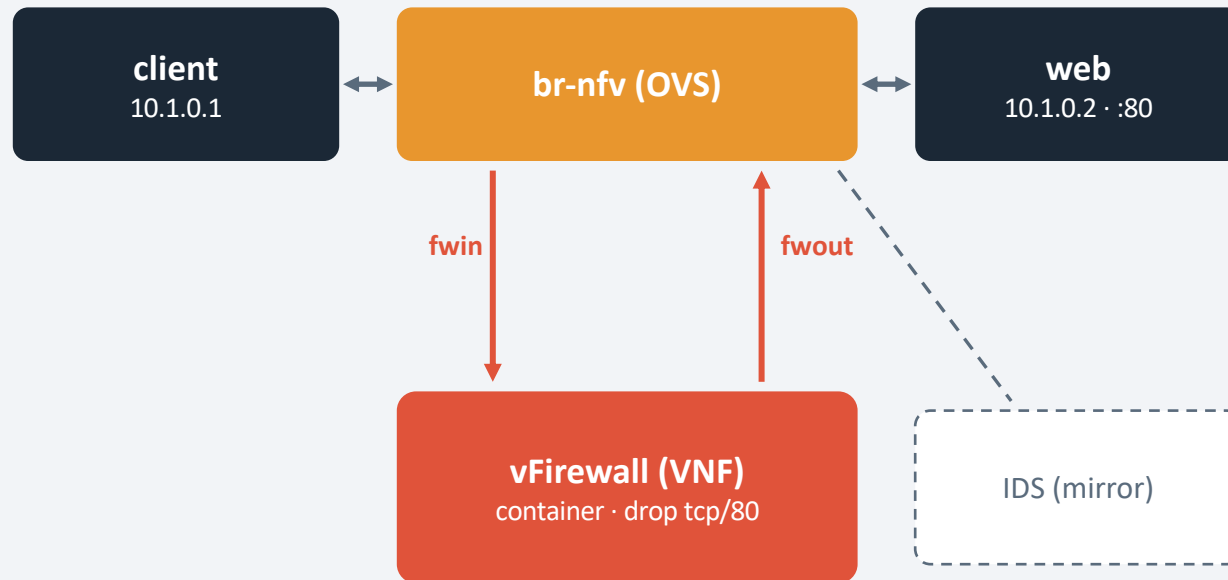
|                   | SDN                                    | NFV                                         |
|-------------------|----------------------------------------|---------------------------------------------|
| Core idea         | Separate control plane from data plane | Separate network functions from hardware    |
| Virtualizes       | Network control (how traffic flows)    | Network functions (what is done to traffic) |
| Runs on           | Controllers + programmable switches    | Standard servers (VMs / containers)         |
| Key standard body | ONF (OpenFlow), IETF                   | ETSI NFV ISG                                |
| Born in           | Campus & data-center research          | Telecom operators                           |
| Example           | Google B4, cloud VPC networks          | Virtual firewall, 5G core                   |

**Neither needs the other, but together:** NFV creates functions anywhere; SDN connects and steers traffic to them.

# Live demo 2: a firewall VNF, chained by flow rules

DEMO 2

SDN steers the traffic. NFV supplies the function.



Only web traffic (tcp/80) is steered into the VNF; ping takes the direct path. Four flow rules, no re-cabling.

1

## Direct path

web page loads

2

## Steer tcp/80 through vFirewall

web blocked · ping still OK

3

## Change policy inside the VNF

web allowed again

4

## Remove the chain

delete rules by cookie

# Demo 2 commands

DEMO 2

Service chaining in four OpenFlow rules

```
demo2.sh

# steer ONLY web traffic through the firewall VNF (cookie tags the chain)
$ ofctl add-flow br-nfv "cookie=0x5fc,priority=200,tcp,in_port=nfv-client,tp_dst=80,actions=output:nfv-fwin"
$ ofctl add-flow br-nfv "cookie=0x5fc,priority=200,in_port=nfv-fwout,actions=output:nfv-web"
$ ofctl add-flow br-nfv "cookie=0x5fc,priority=200,tcp,in_port=nfv-web,tp_src=80,actions=output:nfv-fwout"
$ ofctl add-flow br-nfv "cookie=0x5fc,priority=200,in_port=nfv-fwin,actions=output:nfv-client"
$ docker exec clab-nfv-client curl -m3 http://10.1.0.2          # blocked by the VNF
$ docker exec clab-nfv-client ping -c2 10.1.0.2                # not in the chain: works
# policy change inside the VNF; then remove the chain entirely
$ docker exec clab-nfv-fw /fw.sh allow
$ ofctl del-flows br-nfv "cookie=0x5fc/-1"

# ofctl = sudo ovs-ofctl -O OpenFlow13
```

# 07

## SDN & NFV in industry today

Where these ideas run, and where they are heading

APPLICATION PLANE

CONTROL PLANE

DATA PLANE

# Where SDN & NFV run today

INDUSTRY

You use all four of these every day

1

## Cloud networks

Every AWS/Azure/GCP virtual network (VPC) is SDN: software switches on every server, programmed by the cloud's controllers.

2

## Data-center fabrics

Leaf-spine networks on open hardware running open NOSes like SONiC, configured by automation, not by hand.

3

## Telecom & 5G

The 5G core is a set of cloud-native network functions on standard servers: NFV grown up.

4

## Enterprise WAN

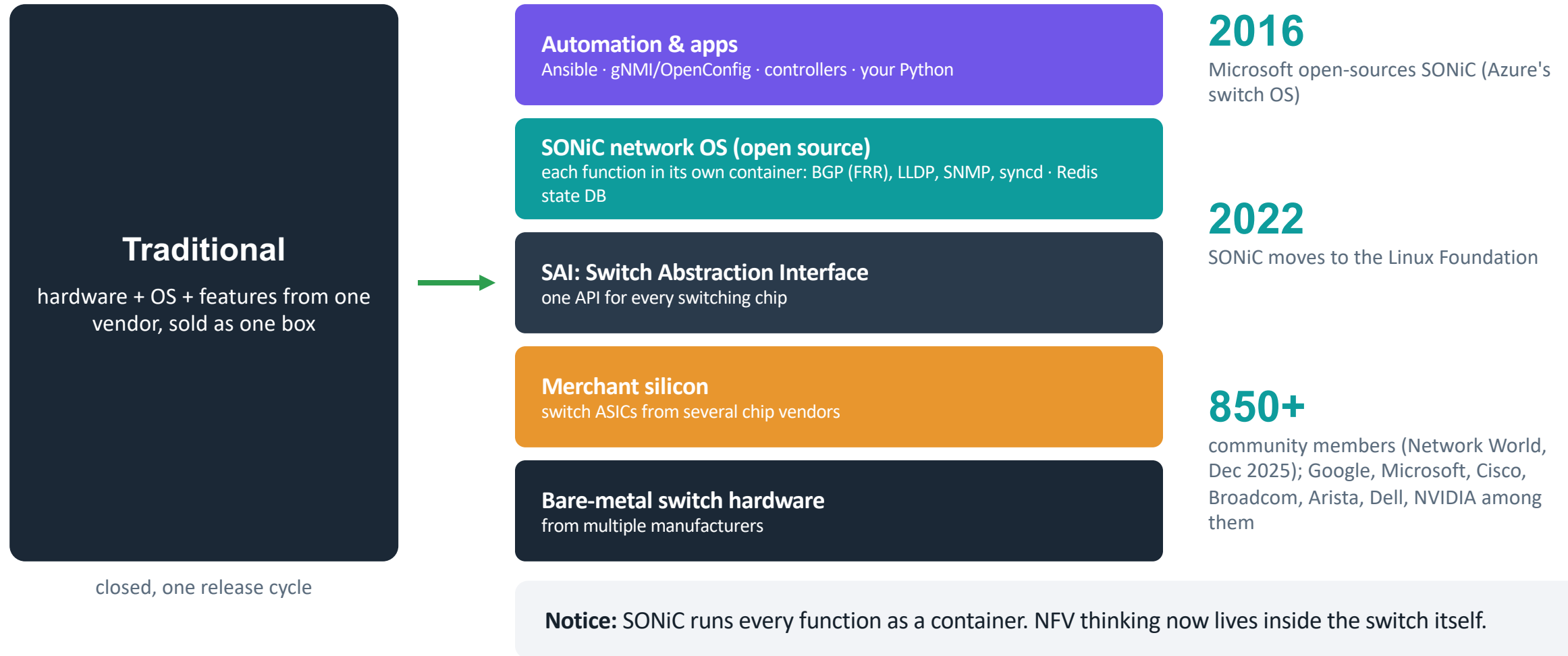
SD-WAN: a central controller picks the best path (MPLS, broadband, 5G) per application for every branch.

**3.1 billion** 5G subscriptions worldwide (Ericsson Mobility Report, June 2026), all served by virtualized, software-defined cores.

# Open networking: the switch, disaggregated

INDUSTRY

The same idea as SDN: separate the layers, open the interfaces



# Where it is heading

INDUSTRY

Four trends worth knowing before your placements

1

## VNF → CNF

Network functions become Kubernetes micro-services. 82% of container users now run Kubernetes in production (CNCF survey, Jan 2026). ETSI is defining a cloud-native "Telco Cloud" for 6G.

2

## Programmable data planes

P4 and SmartNICs/DPUs move packet processing off the CPU and let developers define new pipelines, going beyond fixed OpenFlow tables.

3

## AI networking

AI back-end network switch sales passed front-end sales for the first time in 2Q 2026 (Dell'Oro). Ethernet overtook InfiniBand for AI clusters; Ultra Ethernet 1.0 spec published June 2025.

4

## Autonomous networks

Intent-based, AI-assisted operations. 95 operators and vendors signed TM Forum's Autonomous Networks Manifesto; most target "Level 4" autonomy by 2030.

# Try it on your own

YOUR TURN

Everything from today in one repo: slides, the lab, exercises and resources

## Slides

Today's deck as a PDF

## The lab kit

The exact demo: containerlab topologies, scripts and Dockerfiles

## 7 guided labs

With solutions: be the controller, an ONOS REST app, chain a VNF

## Setup guide

Windows, macOS or Linux, step by step. Free tools only

## Challenges

Write your own controller app, grow the network, send a pull request

## Start in 3 commands

```
git clone .../sdn-nfv-lab
bash scripts/install.sh
bash scripts/up.sh
```

Get everything here

**[github.com/ashokkmr1/sdn-nfv-lab](https://github.com/ashokkmr1/sdn-nfv-lab)**

# Five things to remember

RECAP

1

SDN separates the brain (control plane) from the muscle (data plane) and puts the brain in software with a global view.

2

A switch is a match–action table. OpenFlow is the protocol to fill it; whoever fills it decides what the network does.

3

Apps say WHAT, the controller decides HOW, switches DO it: northbound and southbound APIs.

4

NFV turns network boxes into software (VNFs) on standard servers, managed by MANO: NFVO, VNFM, VIM.

5

SDN and NFV are independent but better together, and they run the cloud, 5G and modern data centers.

# Resources

RECAP

Everything here is free

| Topic         | Where                                                                                                                                                                                                                     |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Textbook      | W. Stallings, Foundations of Modern Networking: SDN, NFV, QoE, IoT, and Cloud (Pearson)                                                                                                                                   |
| Free book     | L. Peterson et al., Software-Defined Networks: A Systems Approach · <a href="http://sdn.systemsapproach.org">sdn.systemsapproach.org</a>                                                                                  |
| OpenFlow spec | ONF OpenFlow Switch Specification 1.3.5 & 1.5.1 · <a href="http://opennetworking.org">opennetworking.org</a>                                                                                                              |
| NFV spec      | ETSI GS NFV 002 (architectural framework) + 2012 NFV white paper · <a href="http://etsi.org">etsi.org</a>                                                                                                                 |
| Labs          | <a href="http://containerlab.dev">containerlab.dev</a> · <a href="http://docs.openvswitch.org/en/latest/tutorials">docs.openvswitch.org/en/latest/tutorials</a> · <a href="http://mininet.org">mininet.org</a>            |
| Controllers   | <a href="http://docs.openstack.org/os-ken">docs.openstack.org/os-ken</a> · <a href="https://github.com/opennetworkinglab/onos">github.com/opennetworkinglab/onos</a> · <a href="http://docs.faucet.nz">docs.faucet.nz</a> |
| Open NOS      | <a href="http://sonicfoundation.dev">sonicfoundation.dev</a> · <a href="http://containerlab.dev/manual/kinds/sonic-vs">containerlab.dev/manual/kinds/sonic-vs</a>                                                         |

# Thank you!

## Questions?

**Ashok Kumar Murthy**

Principal Engineer · Aviz Networks

APPLICATION PLANE

CONTROL PLANE

DATA PLANE